AOSP 进阶丛书 · 第十四册 · ANDROID 13 (API 33)

HAL 驱动级实战

AIDL HAL 五步构建模板（.aidl→Power.cpp→Android.bp→rc+VINTF+SELinux→客户端），三代终局图（legacy→HIDL→AIDL）、从 PowerManager 到内核的完整调用链、GKI 与 vendor_dlkm。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：从 API 到硬件驱动的最后一跳 2
第一章 HAL 三代终局 2
第二章 构建模板：五步走一个新 HAL 3
第三章 调用链：Framework 到 HAL 的完整跳 4
第四章 内核模块与设备树（BSP 视角） 5
第五章 设计决策七问 5
附录 A 验证五步 6
附录 B 快问快答（14 条） 6
附录 C 自检题 7
第六章 hwservicemanager：HIDL 服务注册的中央枢纽 7
第七章 HAL 获取链：getRawServiceInternal 的 transport 分岔 11
第八章 VINTF 清单匹配：libvintf 的兼容矩阵算法 14
第九章 AIDL HAL 端到端：vibrator 全链走读 18
教科书习题集（分级） 22
术语表 25
排障手册：HAL 驱动级实战问题定位 26
HAL 实战核心速查表 28

本册速览

AIDL HAL 构建模板：cc_binary+init_rc+vintf_fragments+vendor:true
参考实现：hardware/interfaces/power/aidl/default/Power.cpp
VINTF 注册与 SELinux 三件套（file/service/sepolicy contexts）
调用链：Framework→ServiceManager→BpHw→HAL 进程
三代终局：legacy(同进程)→HIDL(隔离)→AIDL(大一统)

前言：从 API 到硬件驱动的最后一跳

HAL（Hardware Abstraction Layer）是系统与硬件的边界——书1 § 三讲了三代演进，本册教你**实际写一个**：定义接口→写参考实现→Android.bp→rc/VINTF/SELinux 三件套→客户端调用→端到端验证。以本树 `hardware/interfaces/power/aidl/default/` 的真实代码为模板。

第一章 HAL 三代终局

设计洞察 · 学习目标

1. 用"同进程/跨进程/归一"三关键词区分 legacy、HIDL、AIDL 三代 HAL；2. 说出每代的服务管理设施（无/hwservicemanager/servicemanager）与 binder 设备；3. 理解隔离手段从"IDL 语言"迁移到"分区+VINTF+SELinux"的过程；4. 建立"读码先辨代"的意识：看到 @1.0::IFoo 与 IInterface.descriptor 能立刻分代。



图 1 HAL 三代：legacy(同进程)→HIDL(隔离)→AIDL(大一统)，演进动机与读码区分法

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释HAL 驱动级实战的核心机制与设计决策

- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 核心认知

"隔离"来自 vendor 分区+SELinux，不是来自 IDL 语言。HIDL 引入了进程隔离但代价是双工具链；AIDL 复用主线设施（/dev/binder+servicemanager）后工具链归一——**隔离没丢，只是"隔离的手段"从语言换成了分区+VINTF+策略**。

第二章 构建模板：五步走一个新 HAL

设计洞察 · 学习目标

1. 亲手复现五步流程：.aidl→参考实现→Android.bp→rc/VINTF/SELinux→客户端；2. 逐行解释 aidl_interface 与 cc_binary 的关键属性（vendor_available/init_rc/vintf_fragments/relative_install_path）；3. 独立写出三件套的完整内容并说清各自在运行期的执行点；4. 会用附录 A 的五步验证新 HAL。



图 2 AIDL HAL 五步：定义 .aidl→参考实现→Android.bp→rc+VINTF+SELinux→客户端，以 power/aidl/default 为模板

2.1 Android.bp 逐行注释（真实源码）

text hardware/interfaces/power/aidl/default/Android.bp（节选+讲解注释）

```
aidl_interface {
name: "android.hardware.power", // 模块名：生成四后端（java/cpp/ndk/rust）
vendor_available: true, // 允许 vendor 分区引用
srcs: ["android/hardware/power/*.aidl"],
// 讲解注释①：接口版本冻结在 aidl_api/1, aidl_api/2, ...
// 新增方法必须开新版本目录（stable AIDL 的兼容保证）
}
cc_binary {
name: "android.hardware.power-service.example", // 产物名
relative_install_path: "hw", // → /vendor/bin/hw/
init_rc: [":android.hardware.power.rc"], // 讲解注释②：init 由此找到启动脚本
vintf_fragments: [":android.hardware.power.xml"], // 讲解注释③：VINTF 清单
vendor: true, // 归入 vendor 分区（书1 §七）
shared_libs: [
"libbinder_ndk", // AIDL C++ 的 NDK 库
"liblog",
],
srcs: ["Power.cpp", "main.cpp"],
}
```

2.2 三件套逐行

text rc + VINTF + SELinux（模板）

```
// mysvc.rc
service mysvc-hal /vendor/bin/hw/android.hardware.mysvc-service
class hal // init 的 hal 类启动（书2 §四 late-init 的 early_hal/hal 分支）
user system • group system
seclabel u:r:hal_mysvc:s0 // SELinux 域
// mysvc.xml（VINTF）
<hal format="aidl">
<name>android.hardware.mysvc</name>
<fqname>IMysvc/default</fqname>
</hal>
// SELinux 三件（system/sepolicy/vendor/）
mysvc.te: type hal_mysvc, domain;
file_contexts: /vendor/bin/hw/android.hardware.mysvc-service u:object_r:hal_mysvc_exec:s0
service_contexts: android.hardware.mysvc.IMysvc/default u:object_r:hal_mysvc_service:s0
```

第三章 调用链：Framework 到 HAL 的完整跳

设计洞察 · 学习目标

1. 从 PowerManager.setPowerSaveMode 一路数到内核 sysfs，标出每次进程/域边界；2. 认出链上每段的代码角色（AIDL Proxy/BinderProxy/驱动/Bn Stub/实现类）；3. 能解释为什么 HAL 必须在 vendor 进程（SELinux 域+分区隔离）；4. 把本链与书3 Binder 十跳对齐，知道哪几跳跨进程。

text 全链拆解（以 PowerManager→Power HAL 为例）

```
Java: PowerManager.setPowerSaveMode(true)
→ IPowerManager.Stub.Proxy（AIDL 生成物）
→ BinderProxy.transact（书3 十跳前半）
→ 驱动 → vendor 进程的 binder 线程
→ BnPower::onTransact（AIDL 生成的 native Stub）
→ Power.cpp 的 setMode(bool) 实现
→ 写 sysfs /proc/cpuinfo/…（内核级生效）
```

设计洞察 · 逐段精读：为什么 HAL 在 vendor 进程

书1 Treble 的核心：vendor 代码（SoC 相关）不进 system 分区——升级 system 不必重编 vendor。HAL 进程的 SELinux 域（hal_xxx）与 framework 域隔离，授权通过 service_contexts+allow 规则显式放行。

第四章 内核模块与设备树（BSP 视角）

设计洞察 · 学习目标

1. 说清设备树（dtb/dtbo）与内核模块（.ko，vendor_dlkm）各自描述什么、由谁加载；2. 理解 HAL 与内核驱动的对话面（sysfs/procfs/ioctl）；3. 建立 GKI 分层心智：通用内核不变、厂商差异全走模块；4. 会用本树 device/google/*-kernel 布局指认 Image.lz4/dtbo.img/*.ko 的角色。

HAL 之下是内核世界：**设备树**（dtb/dtbo，硬件拓扑描述——boot.img 分区家族的一员）与**内核模块**（vendor_dlkm，.ko 文件由 insmod 加载）。读码视角：HAL 通过 sysfs/procfs 或 ioctl 与内核驱动对话（书1 § 二的 Android 内核增强）。本树的 `device/google/*-kernel/` 展示了预编译内核+模块的布局（Image.lz4 + dtbo.img + *.ko）。

第五章 设计决策七问

设计洞察 · 学习目标

1. 对每问先给结论再给源码级理由（不背结论背因果）；2. 说出版本冻结、启动时序、VINTF 机器可读性、sysfs 抽象、GKI 分区、进程隔离六个决策的底层动机；3. 能把任一问扩写成面试回答：结论+机制+源码锚点三段式。

- **为什么新 HAL 一律 AIDL？** 工具链归一（一套编译器/调试/线程模型）；HIDL 仅维存量——工程上"少一套"压倒迁移成本。
- **为什么 HAL 接口要版本冻结（aidl_api/1,2,…）？** OTA 兼容性：老版本 framework 必须能调新版本 HAL——stable AIDL 由版本目录保证。
- **为什么 rc 里 class hal 而非 main？** 启动时序：HAL 必须在 framework 服务之前就绪（early_hal 先于 hal 先于 main）。
- **为什么 VINTF 是 XML 而非代码声明？** 兼容匹配需要机器可读：framework 的 compatibility_matrices 与 vendor 的 manifest 在构建期对齐（不匹配=拒绝启动）。

- **为什么 HAL 用 sysfs/procfs 而非直接 mmap 硬件寄存器？** 安全与稳定：内核驱动做权限仲裁+中断处理，HAL 只是"策略到硬件的翻译器"。
- **为什么 vendor_dlkm 独立于 boot？** GKI：通用内核不变，厂商差异全走模块+设备树——内核碎片化治理的镜像版 Treble。
- **为什么 passthrough 模式几乎不推荐？** 同进程=厂商崩溃连坐 framework——进程隔离是 Treble 的底线，性能收益不值得。

附录 A 验证五步

步	命令	通过标准	
①构建	m android.hardware.mysvc	BUILD SUCCESS	
②部署	adb sync 或刷 vendor.img	service list 有 mysvc	
③权限	adb shell ls -Z /vendor/bin/hw/mysvc*	seclabel 正确	
④调用	adb shell cmd mysvc_client --test	返回值正确	
⑤SELinux	logcat -b events |	grep avc	无 denied

附录 B 快问快答（14 条）

- 新 HAL 用什么 IDL？——AIDL（HIDL 冻结）。
- AIDL HAL 走哪个 binder 设备？——/dev/binder（主线域）。
- HIDL 走哪个？——/dev/hwbinder+hwservicemanager（遗留域）。
- AIDL 接口版本冻结在哪？——aidl_api/1,2,… 目录。
- HAL 的 Android.bp 关键四属性？——init_rc+vintf_fragments+vendor+relative_install_path。
- SELinux 三件？——.te+file_contexts+service_contexts。
- VINTF 的作用？——framework 与 vendor 接口版本匹配（不匹配不启动）。
- HAL 进程的 init 类？——class hal（或 early_hal 更早）。
- 为什么 HAL 在 vendor 分区？——升级 system 不必重编 vendor（Treble）。
- 内核模块放哪？——vendor_dlkm 分区（GKI 后）。
- 设备树是什么？——硬件拓扑描述（dtb/dtbo，boot 家族分区）。
- passthrough 是什么？——HIDL 的同进程旁路（几乎不推荐）。
- Sideband 是什么？——相机特权绕 SF 直叠硬件（书8 § 七）。

- 客户端怎么调 HAL？——ServiceManager.getService→Stub.asInterface→方法调用。

附录 C 自检题

- 默写五步走一个 AIDL HAL（含 rc/VINTF/SELinux 内容）。
- 从 PowerManager.setPowerSaveMode 到内核的每跳。
- 为什么 aidl_interface 的版本目录是兼容性保证?
- 读完本树 hardware/interfaces/power/aidl/default 的 Android.bp——每个属性的作用。

第六章 hwservicemanager：HIDL 服务注册的中央枢纽

设计洞察 · 学习目标

1. 说出 hwservicemanager 的三级注册表结构（mServiceMap→PackageInterfaceMap→InstanceMap）与各级键名；2. 走读 ServiceManager::add→addImpl 的完整注册流程并解释 interfaceChain"沿链逐级挂载"的设计；3. 解释重复注册检测、父类卸载、VINTF 父类检查三个防御性设计；4. 用 becomeContextManager 与 tryStartService 解释 hwservicemanager 的内核地位与懒 HAL 触发。

6.0 链路位置：HIDL 世界里的"0 号服务"

第二章的三件套让 HAL 进程能被 init 拉起，但拉起之后第一件事是什么？——**注册**。HIDL 的所有服务（包括 HAL）都挂在 `hwservicemanager` 名下：它自己是 `/dev/hwbinder` 域的第一个进程，通过 `becomeContextManager()` 向内核 binder 驱动登记为 **0 号上下文节点**（handle 0），之后所有 `IServiceManager::add/get` 事务都以它为终点。这一章把第三章调用链中最上游的"注册表"拆开看：它是谁、长什么样、写进去要过几道闸。

先看它自己的启动（`system/hwservicemanager/service.cpp:141-185`，节选讲解）：

cpp system/hwservicemanager/service.cpp:141-185（main 节选+讲解注释）

```
int main() {
// 讲解注释①：HIDL 的 Return<T> 返回值若不检查就丢弃，默认只打日志；
// 但在这里把限制升级为 ERROR——hwservicemanager 自己不能有"吞错误"的代码。
setProcessHidlReturnRestriction(HidlReturnRestriction::ERROR_IF_UNCHECKED);
ProcessState::self()->setCallRestriction(ProcessState::CallRestriction::ERROR_IF_NOT_ONEWAY);
sp<ServiceManager> manager = new ServiceManager();
setRequestingSid(manager, true); // 讲解注释②：请求内核附带调用方 SID（SELinux 上下文）
// ——AccessControl 全靠它判定 canAdd/canGet。
// 讲解注释③：先把自己注册给自己——"manager" 实例的 FQName 是
// android.hidl.manager@1.0::IServiceManager，这是注册表的第 0 条记录。
if (!manager->add(serviceName, manager).withDefault(false)) {
ALOGE("Failed to register hwservicemanager with itself.");
}
...
// 讲解注释④：向内核 binder 驱动登记为本域的上下文管理器——
// 此后 /dev/hwbinder 上 handle 0 的事务全部路由到这里。
```

```
// 这与书3 servicemanager（/dev/binder 域）做了完全对称的事。
IPCThreadState::self()->setTheContextObject(service);
ProcessState::self()->becomeContextManager();
// 讲解注释⑤：属性就绪标志——init.rc 里 HAL 服务的启动条件之一，
// 也是排障第一检查点：getprop hwservicemanager.ready
int rc = property_set("hwservicemanager.ready", "true");
...
}
```

注意 hwservicemanager.rc 里它被标为 critical：它一旦反复崩溃，init 会重启整个系统——注册表丢了，所有 HIDL HAL 都成了孤儿。

6.1 注册表数据结构走读

hwservicemanager 的核心是一张三级 map：**接口全名 →（实例名 → HidlService）**。走读真实头文件。

cpp system/hwservicemanager/ServiceManager.h:110-159（三级注册表+讲解注释）

```
// 讲解注释①：InstanceMap——实例名（如 "default"）到 HidlService 的独占指针。
// HidlService 可能已持有服务 binder（getService() 非空），也可能只是
// "占位"：只有等待注册的 IServiceNotification，getService() 为空。
using InstanceMap = std::map<
std::string, // instance name e.x. "manager"
std::unique_ptr<HidlService>
>;
struct PackageInterfaceMap {
InstanceMap &getInstanceMap();
const InstanceMap &getInstanceMap() const;
// 讲解注释②：lookup 按实例名查找。原注释特意提醒：返回值是"临时引用"，
// 且 getService() 可能为空（该接口有监听者但服务还没来）——
// 这正是懒 HAL 场景下 get() 返回 nullptr 的来源之一。
HidlService *lookup(
const std::string &name);
const HidlService *lookup(
const std::string &name) const;
void insertService(std::unique_ptr<HidlService> &&service);
// 讲解注释③：包级监听表——IServiceNotification::onRegistration 的订阅者，
// 绑定在接口名上而非具体实例，服务一注册就逐个回叫。
void addPackageListener(sp<IServiceNotification> listener);
bool removePackageListener(const wp<IBase>& who);
bool removeServiceListener(const wp<IBase>& who);
void sendPackageRegistrationNotification(
const hidl_string &fqName,
const hidl_string &instanceName);
private:
InstanceMap mInstanceMap{};
std::vector<sp<IServiceNotification>> mPackageListeners{};
};
// 讲解注释④：ACL 裁决器——canAdd/canGet/canList 逐条对照 SELinux
// service_contexts 与 te 规则（第二章"三件套"在这里兑现为运行期检查）。
AccessControl mAcl;
// 讲解注释⑤：最外层 map，键是 FQName 全名（包@版本::接口）。
// 原注释直言：**这个 map 不需要加锁，因为 hwservicemanager 是单线程的**——
```

```
// 用"单线程事件循环"换掉全部互斥开销（对比书3 AIDL servicemanager 同样单线程）。
//
// e.x.
// mServiceMap["android.hidl.manager@1.0::IServiceManager"]["manager"]
// -> HidlService object
std::map<
std::string, // package::interface e.x. "android.hidl.manager@1.0::IServiceManager"
PackageInterfaceMap
> mServiceMap;
```

设计洞察 · 设计洞察：为什么是"单线程+无锁"

hwservicemanager 的 main() 用 Looper::pollAll 事件循环驱动一切（binder 可读事件、客户端计数定时器），所有注册表变更天然串行，于是三级 map 全程免锁。代价是任何回调卡死都会冻结整个 HIDL 域——所以源码里所有"重活"（比如拉起懒 HAL 的属性设置）都用独立线程或异步事件甩出去。这是"以架构约束换性能"的典型案例：约束（单线程）写进了代码注释，成为所有后来者的契约。

6.2 注册事务：add→addImpl

HAL 进程侧调用 `IFoo::registerAsService("default")`，最终以 `hwbinder` 事务抵达这里的 `add`。走读（骨架省略号处为节选）：

cpp system/hwservicemanager/ServiceManager.cpp:311-455（add→addImpl 注册事务，骨架+讲解注释）

```
Return<bool> ServiceManager::add(const hidl_string& name, const sp<IBase>& service) {
bool addSuccess = false;
if (service == nullptr) {
return false; // 讲解注释①：空 binder 直接拒绝
}
auto pidcon = getBinderCallingContext(); // 讲解注释②：从内核取"调用方"PID/UID/SID——
// 权限判定永远基于调用方，不是自己
if (!mAcl.canAdd(IBase::descriptor, pidcon)) {
return false;
}
// 讲解注释③：回查服务对象的 interfaceChain（如 [IFoo@1.0::IFoo, IBase]）。
// 注册是"沿链逐级挂载"：每个父接口名下都能 lookup 到同一服务，
// 客户端的 castFrom 向上转型因此永远本地可达。
auto ret = service->interfaceChain([&](const auto &interfaceChain) {
addSuccess = addImpl(name, service, interfaceChain, pidcon);
});
return addSuccess;
}
bool ServiceManager::addImpl(const std::string& name,
const sp<IBase>& service,
const hidl_vec<hidl_string>& interfaceChain,
const AccessControl::CallingContext& callingContext) {
// 讲解注释④：链条上每一级都要过 ACL——只给 IBase 放行不够，
// service_contexts 必须覆盖叶子接口的 FQName。
for(size_t i = 0; i < interfaceChain.size(); i++) {
if (!mAcl.canAdd(interfaceChain[i], callingContext)) {
return false;
}
}
```

```
const std::string childFqName = interfaceChain[0]; // 链头=叶子接口
// 讲解注释⑤：重复注册检测——用"倒数第二级"（最高父类）查同名实例，
// 命中说明旧服务还活着。只打 WARNING 不拒绝：允许覆盖注册（死亡通知
// 与新注册赛跑时不误杀），但留下新旧 PID 的排障线索。
if (interfaceChain.size() > 1) {
const std::string baseFqName = interfaceChain[interfaceChain.size() - 2];
const HidlService *hidlService = lookup(baseFqName, name);
if (hidlService != nullptr && hidlService->getService() != nullptr) {
LOG(WARNING) << "Detected instance of " << childFqName
<< " registering over " << baseFqName << "..."; // 新旧 PID 见源码
}
}
// 讲解注释⑥：父类一致性——叶子在 VINTF manifest 里还不够，中间父类也必须声明（第八章联动）。
// 讲解注释⑦：正式写入——沿链逐级 insertService/setService，并逐级发注册通知，
// 唤醒所有 IServiceNotification 订阅者（第七章 Waiter 的唤醒端就在这里）。
for(size_t i = 0; i < interfaceChain.size(); i++) {
const std::string fqName = interfaceChain[i];
PackageInterfaceMap &ifaceMap = mServiceMap[fqName];
HidlService *hidlService = ifaceMap.lookup(name);
if (hidlService == nullptr) {
ifaceMap.insertService(
std::make_unique<HidlService>(fqName, name, service, callingContext.pid));
} else {
hidlService->setService(service, callingContext.pid);
}
ifaceMap.sendPackageRegistrationNotification(fqName, name);
}
bool linkRet = service->linkToDeath(this, kServiceDiedCookie).withDefault(false);
return true;
}
```

6.3 懒 HAL：查不到就去"拉"

`ServiceManager::get()`（`ServiceManager.cpp:268-305`）查不到实例时并不直接让客户端失败——它调用 `tryStartService`（`ServiceManager.cpp:246-264`）：起一个 detached 线程，`SetProperty("ctl.interface_start", fqName + "/" + name)`。init 收到这个控制属性后，查找哪个 `.rc` 服务的 `interface` 字段声明了该 FQName，拉起对应进程；服务起来后走本章的 `add` 重新注册，客户端则阻塞在第七章的 `Waiter` 条件变量上等注册通知。这就是**懒 HAL（lazy HAL）**的完整闭环：不用的 HAL 不占内存，用时自动拉起。

> **设计洞察 · 排障衔接**
>
> 懒 HAL 排障三步：1. `lshal` 看 FQName 是否出现（出现了说明 add 已成功）；2. `logcat -s hwservicemanager` 找 "trying to start it as a lazy HAL"——反复出现说明 rc 里没有 interface 条目或服务启动即死；3. `getprop` 过滤 interface_start 无响应说明 init 侧没匹配上 rc。完整命令见文末排障手册场景一/场景四。

本章与既有章节的接口：第三章画的是"调用链"的纵向视角，本章补上了纵向链条第一跳（注册）的内部实现；第二章 rc 模板中 `interface` 属性的用途在本节兑现；第五章"为什么 HAL 用 class hal"的启动时序问题，现在可以补一句——hal 类服务在 `hwservicemanager.ready=true` 之后才值得启动，

否则注册事务无处可投。

第七章 HAL 获取链：getRawServiceInternal 的 transport 分岔

设计洞察 · 学习目标

1. 走读 getRawServiceInternal 并指出 HWBINDER/PASSTHROUGH 两条分支的触发条件；2. 解释 transport 的裁决来源是 VINTF manifest 而非代码；3. 走读 PassthroughServiceManager::openLibs 的 dlopen 约定（-impl 库名与 HIDL_FETCH_ 符号）；4. 对比 binderized 与 passthrough 的进程模型与崩溃边界。

7.0 链路位置：注册的镜像侧

第六章讲了服务怎么"进去"（add），本章讲它怎么"出来"（get）。客户端写 `IFoo::getService("default")`，生成的 HIDL 代理代码最终汇入 libhidl 的总闸函数 `getRawServiceInternal`——它先向 hwservicemanager 问一句"这个接口声明的 transport 是什么"，然后**同一个函数体里分出两条完全不同的路径**：binderized 走跨进程事务，passthrough 走本进程 dlopen。理解这个分岔，就同时理解了 HIDL 的进程模型和第五章"为什么 passthrough 几乎不推荐"的真正含义。

7.1 总闸函数走读

cpp system/libhidl/transport/ServiceManagement.cpp:738-833（获取链总闸，骨架+讲解注释）

```
sp<IBase> getRawServiceInternal(const std::string& descriptor,
const std::string& instance,
bool retry, bool getStub) {
using Transport = IServiceManager1_0::Transport;
sp<Waiter> waiter;
sp<IServiceManager1_1> sm;
Transport transport = Transport::EMPTY;
if (kIsRecovery) {
transport = Transport::PASSTHROUGH; // 讲解注释①：recovery 没有 hwservicemanager，一律同进程
} else {
sm = defaultServiceManager1_1(); // 讲解注释②：hwservicemanager 的 BpHw 代理
Return<Transport> transportRet = sm->getTransport(descriptor, instance);
...
transport = transportRet; // 讲解注释③：关键一问——"这接口是什么 transport？"
} // 答案来自 VINTF manifest（第六章 getTransport：framework→dev
ice 顺序查）
const bool vintfHwbinder = (transport == Transport::HWBINDER);
const bool vintfPassthru = (transport == Transport::PASSTHROUGH);
... // allowLegacy/vintfLegacy：非 enforce 模式的兼容分支
// 讲解注释④：主线分支——binderized，一个带等待的重试循环：
// 先 get，失败建 Waiter 注册通知再重试，阻塞等"服务上线"（懒 HAL 的等待侧）。
for (int tries = 0; !getStub && (vintfHwbinder || vintfLegacy); tries++) {
```

```
if (waiter == nullptr && tries > 0) {
waiter = new Waiter(descriptor, instance, sm);
}
if (waiter != nullptr) {
waiter->reset(); // don't reorder this -- see comments on reset()
}
Return<sp<IBase>> ret = sm->get(descriptor, instance); // 跨进程查注册表（失败略）
sp<IBase> base = ret;
if (base != nullptr) {
// 讲解注释⑤：拿到 binder 还不能直接用——canCastInterface 校验
// 接口版本/哈希兼容（HIDL 接口哈希链；不兼容=注册了错的版本）
Return<bool> canCastRet =
details::canCastInterface(base.get(), descriptor.c_str(), true /* emitError */);
if (canCastRet.isOk() && canCastRet) {
return base; // still needs to be wrapped by Bp class.
}
if (!handleCastError(canCastRet, descriptor, instance)) break;
}
if (vintfLegacy || !retry) break; // legacy 或未要求重试：到此为止
if (waiter != nullptr) {
ALOGI("getService: Trying again for %s/%s...", descriptor.c_str(), instance.c_str());
waiter->wait(true /* timeout */);
}
}
if (waiter != nullptr) { waiter->done(); }
// 讲解注释⑥：旁路分支——passthrough，触发条件：getStub（点名要裸实现）或 VINTF 声明为
passthrough。
if (getStub || vintfPassthru || vintfLegacy) {
const sp<IServiceManager1_0> pm = getPassthroughServiceManager();
if (pm != nullptr) {
sp<IBase> base = pm->get(descriptor, instance).withDefault(nullptr);
if (!getStub || trebleTestingOverride) {
base = wrapPassthrough(base); // 讲解注释⑦：把 dlopen 出的实现再包一层
} // hwbinder 壳，对外仍像远程服务——
return base; // "同进程加速"不改变接口语义
}
}
return nullptr;
}
```

设计洞察 · 逐段精读：transport 由谁定

注意③：客户端进程自己不决定"我要不要跨进程"——它问 hwservicemanager，而 hwservicemanager 查 VINTF manifest（device 侧的 manifest.xml 或 framework 侧 compatibility matrix 配套的 manifest）。也就是说**同一台设备上同一个接口，改一行 XML 就能从跨进程切成同进程，客户端代码零改动**。这是"策略（VINTF 数据）与机制（binder/dlopen）分离"的教科书案例，也是第五章"VINTF 是 XML 而非代码声明"的底层原因。

7.2 passthrough 的加载约定：dlopen 到工厂函数

走读 PassthroughServiceManager——一个**不出进程**的"假 servicemanager"，实现同一个 IServiceManager 接口但 get 变成了文件系统扫描：

cpp system/libhidl/transport/ServiceManagement.cpp:337-466（passthrough 加载，骨架+讲解注释）

```
struct PassthroughServiceManager : IServiceManager1_1 {
static void openLibs(
const std::string& fqName,
const std::function<bool(void*, const std::string&, const std::string&)>& eachLib) {
//fqName looks like android.hardware.foo@1.0::IFoo
size_t idx = fqName.find("::");
...
const std::string prefix = packageAndVersion + "-impl"; // 讲解注释①：库名约定 foo@1.0-impl.so
const std::string sym = "HIDL_FETCH_" + ifaceName; // 讲解注释②：符号约定 HIDL_FETCH_IFoo
constexpr int dlMode = RTLD_LAZY;
void* handle = nullptr;
// 讲解注释③：搜索路径按序 odm→vendor→VNDK-SP（vendor 可用的系统库，书1 §七分区布局）
static std::string halLibPathVndkSp = details::getVndkSpHwPath();
std::vector<std::string> paths = {
HAL_LIBRARY_PATH_ODM, HAL_LIBRARY_PATH_VENDOR, halLibPathVndkSp,
#ifndef __ANDROID_VNDK__
HAL_LIBRARY_PATH_SYSTEM,
#endif
};
for (const std::string& path : paths) {
std::vector<std::string> libs = findFiles(path, prefix, ".so");
for (const std::string &lib : libs) {
const std::string fullPath = path + lib;
if (kIsRecovery || path == HAL_LIBRARY_PATH_SYSTEM) {
handle = dlopen(fullPath.c_str(), dlMode);
} else {
#if !defined(__ANDROID_RECOVERY__) && defined(__ANDROID__)
handle = android_load_sphal_library(fullPath.c_str(), dlMode);
#endif
} // 讲解注释④：sphal——vendor 库经 linker 命名空间隔离加载，防符号表污染
...
if (!eachLib(handle, lib, sym)) {
return; // 返回 false=已加载成功，停止遍历
}
}
}
}
Return<sp<IBase>> get(const hidl_string& fqName,
const hidl_string& name) override {
sp<IBase> ret = nullptr;
openLibs(fqName, [&](void* handle, const std::string &lib, const std::string &sym) {
IBase* (*generator)(const char* name);
*(void **)(&generator) = dlsym(handle, sym.c_str()); // 讲解注释⑤：取工厂函数指针
...
ret = (*generator)(name.c_str()); // 讲解注释⑥：本进程直接 new 出实现对象——
// 没有任何跨进程事务
if (ret == nullptr) { ... return true; } // 此库没有该实例，继续找下一个
using ::android::hardware::details::getDescriptor;
std::string actualFqName = getDescriptor(ret.get());
CHECK(actualFqName.size() > 0);
// 讲解注释⑦：registerReference——向 hwservicemanager 登记"本进程引用了
// 此 passthrough HAL"；懒 HAL 的引用计数与退出判定依赖它（注释⑧：漏掉=进程被误杀）。
registerReference(actualFqName, name);
```

```
return false; // false=停止 openLibs 遍历
});
return ret;
}
```

7.3 两种模式的代价对照

维度	binderized	passthrough
加载方式	init 拉起独立进程（第二章 rc 模板）	客户端进程内 dlopen（-impl.so）
调用路径	每次方法调用一次 hwbinder 事务	本进程直接函数调用
崩溃边界	HAL 崩溃不伤 framework，可由 init 重启	HAL 崩溃即客户端进程崩溃（连坐）
适用接口	几乎所有 AIDL/HIDL HAL	图形（GPU mapper）、recovery、遗留音频编解码
与 VINTF 的关系	manifest 声明 hwbinder transport	manifest 声明 passthrough transport
懒退出支持	LazyServiceRegistrar 按引用计数杀进程	registerReference 通知 hwservicemanager 计数

设计洞察 · 设计洞察：为什么图形走 passthrough

一帧 60Hz 意味着每 16ms 一次调用，binder 事务的拷贝与调度开销无法接受，于是图形/HWC 历史上选择同进程。但注意 Android 13 的边界收缩：AIDL HAL 完全砍掉了 passthrough 概念（AIDL 只有 binderized），连 HIDL 的 default passthrough 服务也在淘汰——"性能豁免"的接口白名单被刻意收窄，因为**崩溃连坐的运维代价是按设备出货量计费的**（第五章第七问的量化版）。

第六章与本章是注册/获取的镜像；第八章揭示 transport 答案的出处（VINTF manifest）与匹配算法；书3 的十跳在 binderized 分支完整适用（把 /dev/binder 换成 /dev/hwbinder 即可）。

第八章 VINTF 清单匹配：libvintf 的兼容矩阵算法

设计洞察 · 学习目标

1. 说出 VINTF 四个数据对象（framework/device 的 manifest 与 compatibility matrix）及两组匹配方向；2. 走读 HalManifest::checkCompatibility 的五段校验（类型/HAL 实例/VendorNDK/SystemSdk/sepolicy+kernel）；3. 走读 MatrixHal::isCompatible 的两级判定（实例级 satisfiesBy、版本级 supportedBy）；4. 解释 optional 标志与"不匹配拒绝启动"的构建期/运行期双重执行点。

8.0 链路位置：transport 答案与启动闸门

第七章客户端问"transport 是什么"、第六章 addImpl 问"父类在不在 manifest"——所有这些问题的答案都出自 `system/libvintf`。VINTF（Vendor Interface）有四个 XML 数据对象：**framework**

manifest（system 侧声明自己提供的接口）、**device manifest**（vendor 侧声明自己提供的接口，即第二章的 vintf_fragments 产物）、**framework compatibility matrix**（system 侧对 device 的要求）、**device compatibility matrix**（device 侧对 system 的要求）。匹配是**双向**的：device manifest 必须满足 framework matrix（系统要的硬件你都得有），framework manifest 必须满足 device matrix（反过来亦然）。运行期执行点是两处：启动早期 `check_vintf` 不通过 init 直接拒启；`servicemanager/hwservicemanager` 在 add/get 时实时查询。

8.1 入口：VintfObject::checkCompatibility

`system/libvintf/VintfObject.cpp:587-640`（骨架讲解）是总入口：取 `getDeviceHalManifest()` 与 `getFrameworkCompatibilityMatrix()` 做第一次 `checkCompatibility`，再取 `getFrameworkHalManifest()` 与 `getDeviceCompatibilityMatrix()` 做第二次，最后在真机上用 `getRuntimeInfo()`（实时内核版本、sepolicy 版本）补第三段校验。任何一段失败，error 字符串就是你在 logcat 里看到的那段 "HALs incompatible. The following requirements are not met:"。下面走读单方向匹配的核心。

8.2 单向匹配：HalManifest::checkCompatibility

cpp system/libvintf/HalManifest.cpp:382-443（单向兼容检查，骨架+讲解注释）

```
bool HalManifest::checkCompatibility(const CompatibilityMatrix& mat, std::string* error,
CheckFlags::Type flags) const {
if (mType == mat.mType) { // 讲解注释①：方向性自检——device manifest 只能
if (error != nullptr) { // 配 framework matrix，同类型配同类型是 bug
*error = "Wrong type; checking " + to_string(mType) + " manifest against "
+ to_string(mat.mType) + " compatibility matrix";
}
return false;
}
// 讲解注释②：第一主战场——逐个 MatrixHal（矩阵条目）核对 manifest 是否满足，
// 收集所有不满足项而不是见错即返（一次给出完整失败清单，排障友好）
auto incompatibleHals = checkIncompatibleHals(mat);
if (!incompatibleHals.empty()) {
if (error != nullptr) {
*error = "HALs incompatible.";
if (mat.level() != Level::UNSPECIFIED)
*error += " Matrix level = " + to_string(mat.level()) + ".";
if (level() != Level::UNSPECIFIED)
*error += " Manifest level = " + to_string(level()) + ".";
*error += " The following requirements are not met:\n";
for (const auto& e : incompatibleHals) {
*error += e + "\n"; // 讲解注释③：逐条列出 required vs provided，
} // 对照排障手册场景二可直接读
}
return false;
}
if (mType == SchemaType::FRAMEWORK) { // 讲解注释④：framework manifest 方向——
if (!checkVendorNdkCompatibility(mat.device.mVendorNdk, framework.mVendorNdks, error)) {
return false; // 多校验 VNDK 快照版本与 SystemSdk 版本
}
```

```
if (!checkSystemSdkCompatibility(mat.device.mSystemSdk, framework.mSystemSdk, error)) {
return false;
}
} else if (mType == SchemaType::DEVICE) {
bool sepolicyMatch = false;
// 讲解注释⑤：device manifest 方向——vendor 的 sepolicy 版本必须落在
// framework matrix 声明的版本区间内（平台 sepolicy 策略的兼容承诺）
for (const auto &range : mat.framework.mSepolicy.sepolicyVersions()) {
if (range.supportedBy(device.mSepolicyVersion)) {
sepolicyMatch = true;
break;
}
}
if (!sepolicyMatch) { ... return false; }
// 讲解注释⑥：内核要求——manifest 里 <kernel> 标签（含 level 属性）与
// framework matrix 的 kernel 条目比对；真机上由 RuntimeInfo 提供实时版本
Level kernelTagLevel = kernel()->level();
if (flags.isKernelEnabled() && shouldCheckKernelCompatibility() &&
kernel()->getMatchedKernelRequirements(mat.framework.mKernels, kernelTagLevel, error)
.empty()) {
return false;
}
}
return true;
}
```

8.3 实例级判定：一个 MatrixHal 怎么算"被满足"

cpp system/libvintf/MatrixHal.cpp:119-150 与 HalManifest.cpp:244-278（两级判定+上游聚合，骨架+讲解注释）

```
// ——上游：HalManifest::checkIncompatibleHals 给 isCompatible 供给数据——
std::vector<std::string> HalManifest::checkIncompatibleHals(const CompatibilityMatrix& mat)
const {
std::vector<std::string> ret;
for (const MatrixHal &matrixHal : mat.getHals()) {
if (matrixHal.optional) {
continue; // 讲解注释⓪：optional="true" 的矩阵条目直接跳过——缺失不算不兼容
}
std::set<FqInstance> manifestInstances;
std::set<Version> versions;
// 讲解注释⓪①：收集 manifest 里同名 HAL 的全部实例与版本（含历史版本追加）
for (const ManifestHal* manifestHal : getHals(matrixHal.name)) {
manifestHal->forEachInstance([&](const auto& manifestInstance) {
manifestInstances.insert(manifestInstance.getFqInstance());
return true;
});
manifestHal->appendAllVersions(&versions);
}
if (!matrixHal.isCompatible(manifestInstances, versions)) {
...
ret.insert(ret.end(), oss.str()); // 拼出 "required: ... provided: ..." 诊断
}
}
return ret;
}
```

```
// ——下游：MatrixHal.cpp:119-150 两级判定——
bool MatrixHal::isCompatible(const std::set<FqInstance>& providedInstances,
const std::set<Version>& providedVersions) const {
// 讲解注释①：矩阵里多个 <version> 条目之间是"或"关系——
// 要求 6.0 或 7.0，manifest 给出其一即可
return std::any_of(versionRanges.begin(), versionRanges.end(), [&](const VersionRange& vr) {
return isCompatible(vr, providedInstances, providedVersions);
});
}
bool MatrixHal::isCompatible(const VersionRange& vr, const std::set<FqInstance>&
providedInstances,
const std::set<Version>& providedVersions) const {
bool hasAnyInstance = false;
bool versionUnsatisfied = false;
// 讲解注释②：第一级——实例级。遍历矩阵要求的每个 interface/instance 组合，
// 要求 manifest 的 FqInstance 集合里逐个找得到对应项
//  （isSatisfiedBy 做包/版本/接口/实例四元组比对，版本区间含 minor 通配）
forEachInstance(vr, [&](const MatrixInstance& matrixInstance) {
hasAnyInstance = true;
versionUnsatisfied |=
!std::any_of(providedInstances.begin(), providedInstances.end(),
[&](const FqInstance& providedInstance) {
return matrixInstance.isSatisfiedBy(providedInstance);
});
return !versionUnsatisfied; // 讲解注释③：任一实例不满足即短路退出
});
if (hasAnyInstance) {
return !versionUnsatisfied; // 有实例要求就按实例级结论收口
}
// 讲解注释④：第二级——矩阵没写实例（如内核 HAL、纯版本要求）时退化为
// 版本级：manifest 的任意版本落入 vr 区间（supportedBy 含 minor 版本兼容）
return std::any_of(
providedVersions.begin(), providedVersions.end(),
[&](const auto& providedVersion) { return vr.supportedBy(providedVersion); });
}
```

上游的 HalManifest::checkIncompatibleHals（HalManifest.cpp:244-278）负责给本节供给数据：跳过 optional="true" 的矩阵条目，把 manifest 里同名 HAL 的全部 FqInstance 与版本收进两个 set，交给 isCompatible，失败则拼出 "required: … provided: …" 的诊断文本。以真实条目为例，framework matrix（hardware/interfaces/compatibility_matrices/compatibility_matrix.current.xml:750-766）对 vibrator 的要求是 optional="true"、version 1-2、IVibrator/default；而本树 hardware/interfaces/vibrator/aidl/default/android.hardware.vibrator.xml 声明的正是 version 2 的 IVibrator/default 与 IVibratorManager/default——版本 2 落在区间 1-2 内，匹配成立。

设计洞察 · 设计洞察：optional 是矩阵的"度"

矩阵不是布尔清单而是带权契约：optional=false 的条目缺失直接拒启（audio、keymint 这类系统命脉）；optional=true 条目（如 vibrator）缺失只影响功能可用性。区间语义（1-2、7.0-1）则把"向后兼容"写进了数据：framework 升级把矩阵从 1.0 推到 2.0 时，仍声明 1-2 区间的老 HAL 不用重编。对照第五章"为什么 VINTF 是 XML"——机器可读的契约才能同时被构建器（assemble_vintf）、启动闸门（check_vintf）和运行期查询（getTransport）三方复用。

本章与既有章节的接口：第二章 vintf_fragments 生成的 XML 在这里是"被匹配的一方"；第七章 transport 分岔与第六章父类检查都调用了 `getTransport`（`system/hwservicemanager/Vintf.cpp:39-70`，先查 framework manifest 再查 device manifest，返回 HWBINDER/PASSTHROUGH/EMPTY 三态）；第九章 vibrator 端到端链路的每一步"注册成功"都隐含本章匹配通过。

第九章 AIDL HAL 端到端：vibrator 全链走读

设计洞察 · 学习目标

1. 走读 vibrator AIDL HAL 的 main.cpp 与 AServiceManager_addService 注册链；2. 走读 AIDL servicemanager 的 addService 与 VINTF 声明检查（meetsDeclarationRequirements）；3. 串起 framework Java→JNI→libvibratorservice→HAL 的完整调用路径；4. 说出 AIDL HAL 与 HIDL HAL 在服务管理上的三点本质区别。

9.0 链路位置：一条活的链

前四章拆的是 HIDL 世界的机制零件，本章把它们重新装回一条真实链路——并换成 Android 13 的新范式 AIDL HAL。对象选 `hardware/interfaces/vibrator/aidl/default/`（第二章五步模板的"活的样板房"）：一部手机的"马达"从 framework 的振动请求到 HAL 实现的每一跳，都能在本树源码里逐行指认。先立全景（每行一次进程/域边界）：

text vibrator 端到端全景（进程边界逐行标注）

```
App: Vibrator.vibrate(...)
→ SystemVibratorManagerService（system_server Java，书3 system_service 域）
→ VibratorManagerService.java（framework 振动调度：并发仲裁/优先级/同步）
→ nativeGetVibratorIds 等 JNI（services/core/jni/…VibratorManagerService.cpp）
→ ManagerHalController / AidlManagerHalWrapper（frameworks/native/services/vibratorservice）
→ waitForVintfService<IVibratorManager>()（先 isDeclared 再等待——第 9.3 节）
→ /dev/binder 事务（注意：不是 hwbinder！AIDL HAL 归主线域）
→ servicemanager（/dev/binder 域 0 号节点，书3 主角，本章 9.2 节）
→ vendor 进程 android.hardware.vibrator-service.example（本章 9.1 节）
→ Vibrator::on(int32_t timeoutMs, callback)（vendor 侧 C++ 实现）
→ （真机上写 sysfs/FF 驱动；default 实现以 log 模拟，第四章的内核边界）
```

对照第三章 Power HAL 的链路，关键差异一目了然：AIDL HAL 不经 hwservicemanager，注册表就住在书3 讲过的那个 servicemanager 里——**两个域、两个注册表、两套三件套，但调用链形态完全同构**。

9.1 服务端：HAL 进程的诞生与注册

cpp hardware/interfaces/vibrator/aidl/default/main.cpp:24-45（HAL 进程 main，全量+讲解注释）

```
int main() {
// 讲解注释①：单线程线程池——vibrator 这种"短平快"接口不需要并发 binder 线程，
// 线程池归零意味着所有事务在主线程串行处理（实现里无需加锁）
ABinderProcess_setThreadPoolMaxThreadCount(0);
// make a default vibrator service
auto vib = ndk::SharedRefBase::make<Vibrator>(); // 讲解注释②：NDK 后端的 C++ 实现，
// SharedRefBase=引用计数的生命周期托管
const std::string vibName = std::string() + Vibrator::descriptor + "/default";
// 讲解注释③：实例名=接口描述符+"/default"
// ——AIDL 的 FQName 约定比 HIDL 简洁
binder_status_t status = AServiceManager_addService(vib->asBinder().get(), vibName.c_str());
CHECK_EQ(status, STATUS_OK); // 讲解注释④：注册失败直接自杀——
// class hal 服务由 init 按 rc 重启
// make the vibrator manager service with a different vibrator
auto managedVib = ndk::SharedRefBase::make<Vibrator>();
auto vibManager = ndk::SharedRefBase::make<VibratorManager>(std::move(managedVib));
const std::string vibManagerName = std::string() + VibratorManager::descriptor + "/default";
status = AServiceManager_addService(vibManager->asBinder().get(), vibManagerName.c_str());
CHECK_EQ(status, STATUS_OK);
// 讲解注释⑤：主线程汇入 binder 线程池等事务——永不返回；
// 进程死亡即 binder 对象消失，servicemanager 的死亡回调会清表（书3 死亡通知）
ABinderProcess_joinThreadPool();
return EXIT_FAILURE; // should not reach
}
```

AServiceManager_addService 只是薄皮（frameworks/native/libs/binder/ndk/service_manager.cpp:34-41）：取 defaultServiceManager()，把名字与 binder 直送 sm->addService(...)——**AIDL HAL 与普通 framework 服务用的是同一个 API 家族**，没有 HIDL 那套 getTransport 预检（预检挪到了 servicemanager 服务端，见下）。

9.2 服务端闸门：AIDL servicemanager 的 addService

cpp frameworks/native/cmds/servicemanager/ServiceManager.cpp:295-349（addService，骨架+讲解注释）

```
Status ServiceManager::addService(const std::string& name, const sp<IBinder>& binder, bool
allowIsol
ated, int32_t dumpPriority) {
auto ctx = mAccess->getCallingContext(); // 讲解注释①：调用方身份（UID/PID/SID）
if (multiuser_get_app_id(ctx.uid) >= AID_APP) {
// 讲解注释②：App UID 一律禁止注册服务——服务注册是平台/厂商特权，
// 这一条把恶意 App 假冒 HAL 的路彻底堵死
return Status::fromExceptionCode(Status::EX_SECURITY, "App UIDs cannot add services");
}
if (!mAccess->canAdd(ctx, name)) {
return Status::fromExceptionCode(Status::EX_SECURITY, "SELinux denial");
} // 讲解注释③：service_contexts+te 的运行期兑现
if (binder == nullptr) { ... }
if (!isValidServiceName(name)) { ... } // 讲解注释④：实例名合法性（含 "/" 格式）
#ifndef VENDORSERVICEMANAGER
// 讲解注释⑤：VINTF 声明检查——只对"vendor 稳定性"的 binder 生效：
```

```
// Stability::requiresVintfDeclaration 判断该对象是否声明了 vintf 稳定性
// （libbinder_ndk 的 AServiceManager_addService 之外，HAL 库会在编译期标记）。
// 普通平台服务不受影响——这就是 AIDL"同一注册表、分级稳定性"的巧妙处
if (!meetsDeclarationRequirements(binder, name)) {
return Status::fromExceptionCode(Status::EX_ILLEGAL_ARGUMENT, "VINTF declaration error");
}
#endif // !VENDORSERVICEMANAGER
if (binder->remoteBinder() != nullptr &&
binder->linkToDeath(sp<ServiceManager>::fromExisting(this)) != OK) { ... }
// 讲解注释⑥：入库——name→Service 结构（binder+allowIsolated+dumpPriority+debugPid）。
// 对比第六章 HIDL：没有 interfaceChain 逐级挂载，AIDL 的向上转型
// 由 binder 驱动+interface_cast 在客户端完成，注册表单级 map 即可
mNameToService[name] = Service {
.binder = binder,
.allowIsolated = allowIsolated,
.dumpPriority = dumpPriority,
.debugPid = ctx.debugPid,
};
auto it = mNameToRegistrationCallback.find(name);
if (it != mNameToRegistrationCallback.end()) {
for (const sp<IServiceCallback>& cb : it->second) {
mNameToService[name].guaranteeClient = true;
// 讲解注释⑦：已有等待者→立刻回叫 onRegistration（对应第七章 Waiter 语义，
// AIDL 侧由 waitForDeclaredService 消费）
cb->onRegistration(name, binder);
}
}
return Status::ok();
}
```

其中⑤的落点是 `isVintfDeclared`（`ServiceManager.cpp:101-118`）：把 `android.hardware.vibrator.IVibrator/default` 拆成包/接口/实例三元组，遍历 device manifest 调 `hasAidlInstance` 查表——VINTF 数据对象与匹配语义复用第八章的 libvintf，只是 HIDL 查 transport、AIDL 查"声明与否"。

9.3 客户端：framework 怎么拿到 HAL

framework 侧三段接力。第一段 Java：`VibratorManagerService.java` 持有 `nativeInit/nativeGetVibratorIds` 等静态方法（`frameworks/base/services/core/java/com/android/server/vibrator/VibratorManagerService.java:177-189`）。第二段 JNI：`nativeInit` 构造 `ManagerHalController`（`com_android_server_vibrator_VibratorManagerService.cpp:80-90`）。第三段 native 获取 HAL，有两个入口——初次连接走 `connectManagerHal`，断线重连走 `tryReconnect`：

cpp frameworks/native/services/vibratorservice/VibratorManagerHalController.cpp:29-41 与 VibratorManagerHalWrapper.cpp:98-105（获取 HAL 两入口，骨架+讲解注释）

```
// ——入口一：初次连接（Controller 构造时调用）——
std::shared_ptr<ManagerHalWrapper> connectManagerHal(std::shared_ptr<CallbackScheduler>
scheduler) {
static bool gHalExists = true;
if (gHalExists) {
// 讲解注释①：waitForVintfService 是 libbinder 的内联模板
// （IServiceManager.h:175-179）——展开后是两步：
```

```
// sm->isDeclared(descriptor+"/default")（问注册表"声明了吗"），
// 声明了才 waitForService（阻塞等待并取 binder）。
// 注意：这里是阻塞版——system_server 启动时愿意等马达 HAL 一小会
sp<Aidl::IVibratorManager> hal = waitForVintfService<Aidl::IVibratorManager>();
if (hal) {
ALOGV("Successfully connected to VibratorManager HAL AIDL service.");
return std::make_shared<AidlManagerHalWrapper>(std::move(scheduler), hal);
}
}
gHalExists = false;
return std::make_shared<LegacyManagerHalWrapper>(); // 讲解注释②：AIDL 版不存在→
} // 降级到 HIDL 1.x 遗留封装（双范式兼容）
// ——入口二：断线重连（Wrapper 内，非阻塞）——
void AidlManagerHalWrapper::tryReconnect() {
sp<Aidl::IVibratorManager> newHandle = checkVintfService<Aidl::IVibratorManager>();
if (newHandle) {
std::lock_guard<std::mutex> lock(mHandleMutex); // 讲解注释③：重连不允许阻塞——
mHandle = std::move(newHandle); // checkVintfService 查不到立即返回空
}
}
```

设计洞察 · 逐段精读：isDeclared 与 checkService 为何拆两步

waitForService 系查不到会**阻塞等待**，checkService 系查不到**立即返回空**。framework 对"可选硬件"（vibrator 在 matrix 里是 optional=true）的组合拳是：初次连接用 waitForVintfService（启动期可等，但 isDeclared 前置保证不等"根本没声明"的 HAL），重连用 checkVintfService（运行期绝不阻塞）。这组 API 三兄弟（checkService/getService/waitForDeclaredService）的选择，本质是"这个硬件在矩阵里 optional 还是必选、现在处于启动期还是运行期"的代码投影——第八章的数据决策，在这几行落地。

9.4 抵达硬件：HAL 实现与回调

cpp hardware/interfaces/vibrator/aidl/default/Vibrator.cpp:49-62（on 实现，全量+讲解注释）

```
ndk::ScopedAStatus Vibrator::on(int32_t timeoutMs,
const std::shared_ptr<IVibratorCallback>& callback) {
LOG(VERBOSE) << "Vibrator on for timeoutMs: " << timeoutMs;
if (callback != nullptr) {
// 讲解注释①：回调必须异步——on() 要立即返回（binder 线程是宝贵的），
// 起独立线程睡眠 timeoutMs 后再回头调客户端的 onComplete
std::thread([=] {
LOG(VERBOSE) << "Starting on on another thread";
usleep(timeoutMs * 1000);
LOG(VERBOSE) << "Notifying on complete";
if (!callback->onComplete().isOk()) {
LOG(ERROR) << "Failed to call onComplete";
}
}).detach();
}
return ndk::ScopedAStatus::ok(); // 讲解注释②：default 实现到此为止；真机版本在这里
// 写 sysfs/ff-memless 接口（第四章的内核边界）
}
```

值得停一步：`callback` 是 framework 传来的另一个 binder 对象——**HAL 到 framework 的反向调用**。也就是说完整的振动生命周期里，`/dev/binder` 上跑了两个方向的事务：正向 on、反向 onComplete。HAL 进程若线程池为 0（本例正是），反向回调将占用其主线程——框架层为什么把振动合成/调度全收进 VibratorManagerService 而让 HAL 保持"哑"，这里给出了答案：HAL 越哑，反向依赖越少，binder 线程越安全（第五章设计决策的又一注脚）。

> **设计洞察 · 设计洞察：AIDL HAL 与 HIDL 的三点本质区别**
> 1. 注册表归一：AIDL HAL 用 /dev/binder+servicemanager（书3 设施全复用），HIDL 用 /dev/hwbinder+hwservicemanager——少一个域就少一整套运维工具；2. 稳定性分级：AIDL 用 Stability 注解区分 platform/vendor 稳定性，VINTF 检查只对声明 vendor 稳定性的对象生效，普通服务与 HAL 共存一个注册表互不干扰（第六章 HIDL 则人人必须过 VINTF）；3. 没有 passthrough：AIDL 砍掉同进程旁路（第七章），进程隔离不再有性能豁免口子。三点合起来就是第五章第一问"工具链归一"的源码证据。

本章与既有章节的接口：第二章五步模板的 rc/VINTF/SELinux 三件套在 9.1-9.2 兑现为运行期检查；第三章调用链的每跳均可在 9.0 全景里对号；第八章的 optional 语义决定 9.3 用哪个 API 取 HAL；第六章的双级注册表与本章的单级 ServiceMap 对比，是理解"interfaceChain 有没有必要"的最佳素材。

教科书习题集（分级）

第一级：简答题（检验概念理解，共 10 题）

- **1. HAL 三代各用什么服务管理设施与 binder 设备？**

  参考答案要点：legacy 无服务管理（dlopen 同进程）；HIDL 用 hwservicemanager+/dev/hwbinder；AIDL 用 servicemanager+/dev/binder——三代演进即注册表与隔离手段的演进。

- **2. hwservicemanager 的注册表是几级结构？各级键是什么？**

  参考答案要点：三级：mServiceMap（FQName 全名）→PackageInterfaceMap（实例名如 "default"）→HidlService（binder+监听者）。单线程设计因此全程免锁。

- **3. 什么是 interfaceChain？注册时为什么沿链逐级挂载？**

  参考答案要点：接口继承链（如 [IFoo, IBase]）；沿链逐级 insertService 使每个父接口名下都能 lookup 到该服务，客户端 castFrom 向上转型无需再跨进程。

- **4. transport（HWBINDER/PASSTHROUGH）由什么决定？客户端从哪里查询？**

  参考答案要点：由 VINTF manifest 声明决定；客户端经 hwservicemanager 的 getTransport 查询（Vintf.cpp：先查 framework manifest 再查 device manifest），策略与机制分离。

- **5. passthrough 库的文件名与工厂符号约定是什么？**

参考答案要点：库名 <包@版本>-impl.so（如 android.hardware.foo@1.0-impl.so）；符号 HIDL_FETCH_<接口名>；openLibs 按路径序扫描后 dlsym 调用工厂函数构造实现。

- **6. android_load_sphal_library 与普通 dlopen 的区别?**

参考答案要点：sphal 经 linker 命名空间从 vendor 镜像加载 same-process HAL 库，隔离 vendor .so 与系统符号表；recovery/system 路径才退回普通 dlopen。

- **7. AIDL servicemanager 的 addService 比 HIDL 的 add 多/少了哪些检查?**

参考答案要点：多了 App UID 一律拒绝、isValidServiceName；VINTF 检查按 Stability 分级（仅 vendor 稳定性对象需声明）；少了 interfaceChain 逐级挂载——AIDL 转型由客户端 interface_cast 完成。

- **8. checkService、getService、waitForDeclaredService 语义差异与适用场景?**

参考答案要点：checkService 立即返回（可空）；getService/waitForService 阻塞等待；waitForDeclaredService 先 isDeclared 再等待——可选硬件用 check 系，必选硬件用 wait 系。

- **9. MatrixHal::isCompatible 的两级判定是什么?**

参考答案要点：矩阵写了实例→实例级：要求的每个 interface/instance 都要在 manifest 的 FqInstance 集合里 isSatisfiedBy；矩阵没写实例→版本级：manifest 版本落入 versionRange（supportedBy）。

- **10. 懒 HAL 从 getService 到服务上线的完整触发链?**

参考答案要点：get 查无→tryStartService 设置 ctl.interface_start 属性→init 匹配 rc 的 interface 字段拉起进程→HAL add 注册→onRegistration 回叫唤醒 Waiter→客户端拿到 binder。

第二级：分析题（检验机制理解，共 7 题）

- **11. 走读 getRawServiceInternal：画出 HWBINDER 分支从 getTransport 到返回 Bp 代理的每一步，标注进程边界与阻塞点。**

参考答案要点：①hwservicemanager 问 transport；②循环 sm->get 跨进程查表；③canCastInterface 本地校验版本；④失败建 Waiter 注册通知阻塞；⑤成功返回后由生成的代理层包成 BpHw；阻塞点仅在 Waiter.wait。

- **12. 失败推演：device manifest 漏写某 HAL 条目，注册、获取、启动闸门各在哪一步失败?**

参考答案要点：注册：registerAsServiceInternal 查 transport 非 HWBINDER 报 "must be in VINTF manifest"；获取：getTransport 返回 EMPTY 走 legacy 或失败分支；启动：PRODUCT_ENFORCE_VINTF_MANIFEST 下 check_vintf 拒启。

- **13. 失败推演：hwservicemanager 崩溃重启后，HIDL HAL 与 AIDL HAL 各经历什么?**

参考答案要点：rc 标 critical——反复崩溃会整机重启；单次重启后注册表清空，HIDL HAL 因死亡通知被 init（class_restart hal）重启重注册，客户端凭死亡通知重建；AIDL HAL 走 /dev/binder+servicemanager，完全不受影响——这是 AIDL 可用性优势的极端展示。

- **14. addImpl 对重复注册只打 WARNING 不拒绝，分析这个取舍。**

参考答案要点：利：死亡通知与新注册赛跑时不误杀新实例，恢复更快；弊：配置错误（服务装两份）静默覆盖，仅靠新旧 PID 日志排查。可改进：对同 PID 覆盖拒绝、跨 PID 覆盖告警+metric。

- **15. hwservicemanager 单线程无锁，什么调用会拖垮它？tryStartService 为何起 detached 线程?**

参考答案要点：任何回叫（onRegistration/linkToDeath）卡死都会冻结整个 HIDL 域——所有 add/get 排队；SetProperty 是同步 binder 调用且可能在锁内被触发，detached 线程避免事件循环反堵。

- **16. passthrough 的 registerReference 若漏掉，懒 HAL 会发生什么?**

参考答案要点：hwservicemanager 不知道有客户端，引用计数恒 0，LazyServiceRegistrar 判定"无人使用"杀进程——库被 dlclose/进程退出后客户端持有悬空指针，同进程直接崩溃。

- **17. 跨册串联：binderized HAL 的一次方法调用对应书3 十跳的哪几跳？hwbinder 与 binder 在驱动层有何不同?**

参考答案要点：客户端 Proxy→BinderProxy.transact→驱动→HAL 进程 Bn::onTransact——十跳的前半完全一致；差异在设备节点与上下文管理器：hwbinder 是独立域、0 号节点是 hwservicemanager，两域注册表互不可见。

第三级：设计题（检验综合能力，共 3 题）

- **18. 设计 lights 的 AIDL HAL：给出完整五件套与验证点。**

参考答案要点：android.hardware.lights.ILights/default；Lights.cpp 写 /sys/class/leds/*；Android.bp 照抄第二章 power 模板改四属性；rc class hal+interface 条目；XML fqname+service_contexts hal_lights_default；验证 lshal+cmd lights 测试+logcat avc。

- **19. 设计 HAL 健康监控：检测死亡、重启循环、慢调用。**

参考答案要点：死亡：linkToDeath/IServiceNotification+death 事件上抛；重启循环：init 属性 svc.<name>.restarting 统计或 logcat 匹配 Service 崩溃窗口；慢调用：binder 事务时间戳（dumpsys binder --stats）+HAL 内 SLOW 子线程 watchdog；产物建议走 dumpsys 自定义接口输出。

- **20. 设计 IVibrator 从 v1 到 v2 的版本升级：新增 setAmplitudeFifo 的兼容迁移。**

参考答案要点：aidl_api/android.hardware.vibrator/2 冻结 v1+新增 v2 目录；HAL 同时实现 v1（默认行为）与 v2；matrix 升 version 1-2→1-3 之前 framework 先用 getInterfaceVersion 探测；老 HAL 在新 framework 下走 v1 分支——stable AIDL 的"新增不改旧"原则。

术语表

术语	解释
AccessControl	hwservicemanager 的 ACL 裁决器：canAdd/canGet/canList 逐条对照 SELinux service_contexts 与 te 规则
AIDL HAL	用 stable AIDL 定义、走 /dev/binder+servicemanager 的新范式 HAL，Android 13 主线方向
aidl_api	stable AIDL 的版本冻结目录（1、2、…），"新增不改旧"是 OTA 兼容的根基
Binderized	HIDL 跨进程模式：HAL 独立成进程，每次方法调用是一次 hwbinder 事务
becomeContextManager	向 binder 驱动登记为本域 0 号上下文节点——servicemanager 家族"自立门户"的仪式
compatibility matrix	兼容矩阵：framework/device 互相提出的接口要求，含 optional 标志与版本区间
ctl.interface_start	init 控制属性，懒 HAL 的启动触发器：属性值即 FQName/实例名
FQName	包@版本::接口 全限定名（如 android.hardware.foo@1.0::IFoo），HIDL 注册表第一级键
HIDL_FETCH_IFoo	passthrough 库的 C 工厂符号约定：dlsym 找到它即能同进程构造实现对象
HidlService	hwservicemanager 注册表叶节点：FQName+实例名+可选 binder+注册监听者集合
hwbinder	/dev/hwbinder，HIDL 专用 binder 设备域，与 /dev/binder 双域隔离
hwservicemanager	/dev/hwbinder 域 0 号节点：HIDL 服务注册表，单线程事件循环、全程无锁
interfaceChain	接口继承链（[IFoo, IBase]）；HIDL 注册沿链逐级挂载，支撑客户端向上转型
lazy HAL	懒 HAL：默认不启动，客户端首次 getService 时经 ctl.interface_start 拉起
libvintf	VINTF 数据结构与匹配算法库：manifest/matrix 解析、checkCompatibility、VintfObject 统一入口
manifest	清单：framework/device 各一份，声明"我提供这些接口"及各自 transport
passthrough	HIDL 同进程模式：客户端进程 dlopen -impl.so 直接调用，无进程隔离（逐步淘汰）

术语	解释
registerReference	passthrough 客户端向 hwservicemanager 登记引用——懒 HAL 引用计数与退出判定的依据
sphal	same-process HAL：vendor 共享库经 linker 命名空间隔离加载的机制
Stability	AIDL 稳定性分级：声明 vendor 稳定性的对象在 servicemanager 注册时受 VINTF 检查
transport	传输模式三态 HWBINDER/PASSTHROUGH/EMPTY，由 VINTF manifest 裁决、getTransport 查询
VINTF	Vendor Interface：manifest+matrix 构成的机器可读兼容契约，启动闸门+运行期查询双执行
vintf_fragments	Android.bp 属性：HAL 模块向 device manifest 注入 XML 片段，构建期自动汇总
懒退出	最后一个客户端注销后自动杀死 HAL 进程（LazyServiceRegistrar+引用计数）

排障手册：HAL 驱动级实战问题定位

定位次序：先辨代（HIDL 查 hwservicemanager、AIDL 查 servicemanager）→查注册表→查 VINTF→查 SELinux→查进程本身。以下六场景覆盖九成现场问题。

场景一：HAL 服务未注册（lshal 查不到）

- **症状**：客户端 getService 返回空或阻塞；`lshal` 输出里没有目标 FQName。
- **定位命令**：
  - `adb shell lshal | grep -i vibrator`（HIDL/AIDL 全列）
  - `adb shell getprop hwservicemanager.ready`（HIDL 域：应为 true）
  - `adb shell ps -A | grep hal`（进程在不在）
  - `adb shell logcat -d -s hwservicemanager ServicerManager init`
- **根因**：rc 缺失或 class 错误（hal 未被启动）；进程启动即崩（Android.bp 依赖缺失）；AIDL HAL 注册名与客户端查询名不一致。
- **修复**：补 rc 并确保 `class hal`；对照第二章模板补 `init_rc` 与 `relative_install_path: "hw"`；核对 `Vibrator::descriptor + "/default"` 拼写。

场景二：VINTF 校验失败

- **症状**：logcat 出现 "must be in VINTF manifest in order to register/get" 或启动闸门报 "HALs incompatible. The following requirements are not met:"。
- **定位命令**：

- adb shell cat /vendor/etc/vintf/manifest.xml（确认条目）
- adb shell lshal --neat（对照实际注册与声明）
- check_vintf --manifest ... --matrix ...（主机侧复现）
- adb shell dmesg | grep -i vintf

- **根因**：vintf_fragments 未加进 Android.bp；XML 的 name/version/fqname 三元组与实际注册不符（版本区间错最常见）；HIDL 父类接口未随子类一起声明（第六章 addImpl 父类检查拒绝注册）。
- **修复**：补 XML 片段并用 `assemble_vintf` 预校验；版本写区间（1-2）而非单点；沿 interfaceChain 补齐每一级声明。

场景三：getService 阻塞不返回（懒 HAL 死等）

- **症状**：客户端卡在 getService 永不返回；hwservicemanager 日志反复打印 "is not registered, trying to start it as a lazy HAL"。
- **定位命令**：
  - adb shell logcat -d | grep -E "trying to start it as a lazy HAL"
  - adb shell getprop | grep ctl.interface_start（无响应=init 没接住）
  - adb shell grep -r "interface" /vendor/etc/init/*.rc
- **根因**：rc 里没写 `interface` 字段（init 不知道 ctl.interface_start 该拉谁）；或服务启动后立即死（端口/权限），拉起-死亡循环。
- **修复**：rc 服务条目补 `interface <FQName>`；先解决场景一的启动即死问题；临时验证可用 `ctl.interface_start` 手动触发对照。

场景四：passthrough 崩溃循环或 dlopen 失败

- **症状**：logcat 报 "Failed to dlopen xxx-impl.so" 或 "could not find symbol HIDL_FETCH_xxx"；使用方进程连带崩溃。
- **定位命令**：
  - adb shell ls /vendor/lib*/hw/ | grep impl（库存在吗）
  - adb shell nm -D /vendor/lib64/hw/xxx-impl.so | grep FETCH（符号在吗）
  - adb shell cat /proc/self/ldlibs 2>/dev/null; adb shell lshal --init-release（命名空间排查）
  - adb shell logcat -b crash
- **根因**：-impl.so 未装进 vendor 分区或位宽不对（lib 与 lib64）；`HIDL_FETCH_` 符号被编译器裁掉（非 C linkage/未加 extern "C"）；sphal 命名空间未放行该库依赖。

- **修复**：cc_library_shared 补 `vendor: true` 与正确位置；工厂函数包 extern "C"；对照 `android/hardware/.../impl` 库的 lib dependencies 检查 sphal namespace 配置。**根治**：迁移到 binderized（第五章第七问）。

场景五：SELinux denied（起不来或调用被拒）

- **症状**：HAL 进程起来但 add 失败，或客户端 transact 报权限错误；events 日志见 avc denied。
- **定位命令**：

  - `adb shell logcat -b events | grep avc`（注意管道符转义场景）

  - `adb shell dmesg | grep avc`

  - `adb shell ls -Z /vendor/bin/hw/`（标签对不对）

  - `adb shell ps -Z | grep <hal>`

- **根因**：file_contexts 给可执行文件打了错标签；service_contexts 缺 FQName 条目（canAdd/canGet 被 AccessControl 拒）；te 文件缺 allow（hal 域无 binder 调用权限）。
- **修复**：按第二章三件套逐一对齐：exec 标签→域自动切换→service_contexts 放行 add/get→te 补 allow；audit2allow 生成草稿后人工审读再合入。

场景六：hwbinder 事务超时/线程耗尽

- **症状**：调用偶发超时；HAL 进程 CPU 低但响应慢；dumpsys 显示 binder 线程全部繁忙。
- **定位命令**：

  - `adb shell dumpsys -l | grep -i binder`；`adb shell dumpsys binder --stats`（需 root 环境）

  - `adb shell cat /sys/kernel/debug/binder/proc/<pid>`（事务栈）

  - `adb shell ps -T -p <hal_pid> | head -20`（binder 线程数）

- **根因**：HAL 实现把长耗时操作（等中断、sleep）放在 binder 线程上同步执行，线程池耗尽后新事务排队；on() 的回调若在 binder 线程内直接回叫（对比第九章 Vibrator::on 起独立线程的正例）形成往返死等。
- **修复**：长操作移到 HAL 内部工作线程（HAL 接口立即返回）；回调一律异步；线程池按接口并发度调 `setMaxThreadPoolCount`；用 systrace 验证事务时长分布。

HAL 实战核心速查表

机制	一句话	源码锚点	所在册
五步模板	.aidl→Power.cpp→Android.bp→rc+VINTF+SELinux→客户端	power/aidl/default/	本册

机制	一句话	源码锚点	所在册
aidl_interface	自动生成四后端（java/cpp/ndk/rust）	Soong 模块	本册
版本冻结	aidl_api/1,2,... 目录（stable AIDL）	hardware/interfaces	本册
VINTF	framework matrix vs vendor manifest 对齐	hardware/interfaces/*/...xml	本册
SELinux 三件	.te+file_contexts+service_contexts	system/sepolicy/vendor/	本册
init class	class hal（或 early_hal 更早启动）	init.rc	本册
vendor_dlkm	GKI 厂商内核模块分区	boot.img 家族	本册
passthrough	HIDL 同进程旁路（几乎不推荐）	HIDL 遗留	本册